Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC3745: Testing
Actividad 1: Unit Testing & Coverage

Alison Fernandez Blanco & Maximiliano Narea

Miércoles 11 de marzo de 2026

1 Objetivos

Esta actividad evalúa la aplicación de los contenidos vistos en clases sobre **pruebas unitarias**, **test doubles** y **cobertura de código**.

Al finalizar, se espera que puedas:

- Diseñar y ejecutar **pruebas unitarias efectivas** para cubrir tanto los caminos de éxito como los de error.
- Tomar decisiones informadas sobre cuándo **aislar dependencias** mediante **mocks** (u otros doubles) para contar con tests que sigan el principio **FIRST**.
- Utilizar herramientas de **cobertura** para detectar código no cubierto y mejorar tu suite de tests hasta alcanzar **100%** en las clases indicadas.

2 Descripción

En esta actividad debes implementar pruebas unitarias para dos clases: `PricingService` y `CheckoutService`. **No debes modificar el código original** provisto en la carpeta `src/`. El objetivo es lograr la cobertura completa únicamente mediante pruebas de las siguientes clases:

- `PricingService`: Implementa pruebas unitarias para alcanzar **100% de coverage** en esta clase. `PricingService` calcula el total a pagar de un carrito e incluye: subtotal, aplicación de cupón, impuestos según el país y el costo de envío según el país y el monto.
- `CheckoutService`: Implementa pruebas unitarias para alcanzar **100% de coverage** en esta clase. `CheckoutService` orquesta el proceso de checkout y depende de componentes externos (por ejemplo, el pago y el envío por correo). Por lo tanto, debes aplicar técnicas para que tus pruebas **no dependan de servicios externos** (red/BD/correos reales) y cumplan con el principio **FIRST**.

  Sugerencia: Revisa la biblioteca `unittest.mock` de Python para crear mocks.

Principio FIRST: Tus tests deben ser **Fast, Independent, Repeatable, Self-validating, Timely**.

3 Código Base

GitHub Classroom. Para realizar la actividad, debes unirte al repositorio de GitHub Classroom del curso y aceptar la *Actividad 1*.

- *Link Classroom:* `https://classroom.github.com/a/AG7uuOyW`

Al aceptar la actividad, se creará un repositorio con la siguiente estructura:

```
src/models.py
src/checkout.py
src/pricing.py
tests/__init__.py
tests/test_checkout_service.py
tests/test_pricing_service.py
Actividad1.ipynb
.coveragerc
```

Los archivos en `tests/` no contienen pruebas unitarias (solo contienen un esqueleto), por lo que debes implementar tus pruebas siguiendo la descripción anterior.

Evaluación. El archivo `Actividad1.ipynb` incluye celdas para ejecutar los tests y medir la cobertura. **No modifiques las últimas celdas del notebook**, ya que se utilizan para la autocorrección.

También puedes ejecutar las pruebas y calcular cobertura desde consola:

```
coverage run --rcfile=.coveragerc -m unittest discover -s tests
```

Para ver qué líneas y ramas fueron cubiertas, y el porcentaje total alcanzado:

```
coverage report -m
```

Importante: La calificación se calcula en función del **porcentaje de cobertura total** obtenido en tu entrega. Este valor considera **statement coverage** y **branch coverage** de las clases mencionadas. En el notebook encontrarás una referencia del cálculo de la nota.

4 Entregable, Fecha de Entrega y Penalizaciones

Debes renombrar el archivo `Actividad1.ipynb` con tu número de alumno/a (por ejemplo, `17626465.ipynb`) y subir al repositorio:

- el notebook renombrado, y
- los archivos de tests que modificaste en `tests/`.

4.1 Fechas de Entrega y Penalizaciones

- **Fecha de entrega:** Viernes 13 de marzo hasta las 23:59.
- **Atraso menor a 1 hora (-0,5 puntos):** Sábado 14 de marzo desde 00:00 hasta las 00:58.
- **Atraso de 24 horas (-1 punto):** Sábado 14 de marzo desde 00:59 hasta las 23:59.

- **Atraso de 48 horas (-2 puntos):** Domingo 15 de marzo desde 00:00 hasta las 23:59.

Nota: Los descuentos por atraso se aplican a la nota final de la actividad.